

PrismatAI Training Plan — V3 Ready

Date: 2026-03-09 **Status:** FINAL — reviewed (6 external reviews + meta-review), ready to implement

Approach: Track A — Supervised learning on human replays first; iterative self-play in a future phase **Author:** Surfinite + Claude Code

Key Verification Items

These are facts the plan depends on. If any are wrong, assumptions about data scale, feasibility, or success criteria may need revision.

- **V1 — Usable human replay count (validated, final):** Full balance validation (cost + rarity against current `cardLibrary.json`) completed across **all 203,602 codes** on March 7, 2026. Results: **154,061 balance-passed**, 48,707 failed, 834 fetch errors (S3 404s). After applying the full quality filter (both players ≥ 1500 rating at game time, rating change $\neq 0$ to exclude friendly/unrated games): **102,697 usable training games**. 740 unique players contribute. Post-patch pass rate: $\sim 97.8\%$. Pre-patch pass rate: $\sim 57.7\%$ — lower because many pre-patch games used unit costs/rarities that have since changed. This is the authoritative training set size.
- **V2 — Competitive unit count:** The authoritative unit list is `bin/asset/config/valid_units.json` and contains exactly **116 units**: 11 base set units (present in every game) and 105 dominion/random set units (the ranked competitive pool). This list was derived from the live game's unit selection screen, March 2026. The `cardLibrary.json` contains 161 entries but 45 of those are deprecated, event-mode, or removed units that no longer appear in competitive play. Any feature schema must be built on the **116-unit list**, not the raw `cardLibrary` count.
- **V3 — Balance validation definition:** A replay is considered balance-valid if its unit costs **and unit rarities (supply limits)** match the current `cardLibrary.json`. The validation script (`validate_db_codes.py`) has been extended to check both `buyCost` and `rarity` for every unit in every replay — it is fully general and catches changes from any era of the game's history.

Decision: validate all replays at training time. The January 14, 2019 cutoff was previously used as a fast filter (post-patch games assumed valid without checking). This approach is fragile — it relies on the boundary being correct and complete, and does not catch rarity/supply changes that happened at other points. Instead, every replay in the training set must pass the full cost+rarity validation before use, regardless of timestamp.

Rarity audit results (March 2026): A sample of 6,308 replays across 133 monthly buckets identified **17 units with historical rarity changes**:

- **Bombarder:** normal $\rightarrow$ rare (supply 20 $\rightarrow$ 4), affected replays dated July 2018-March 2019. Approximately 544 training-eligible replays will be flagged.
- **16 other units:** mostly trinket $\rightarrow$ normal or trinket $\rightarrow$ legendary transitions, clustered around November 2015. These predate most high-rated play and will affect a smaller number of training-eligible replays.

Rarity mismatches are reported as `issue: "rarity_changed"` alongside the existing `issue: "cost_changed"` failures. Both cause a replay to fail validation and be excluded from the training set. Audit data: `c:\libraries\prismata-replay-parser\rarity_audit_results.json`.

- **V4 — JS engine accuracy:** The JavaScript engine is a transpilation of the original AS3 engine and is considered authoritative. Previously validated against replay data (100% pass rate on 500 replays), but **revalidation required** — the JS engine code was modified during recent integration work. Run a fresh 500-replay validation before Phase 2 extraction begins. During Phase 2a extraction, every replay will be validated as it passes through the engine. Any failures will be logged and excluded.
- **V5 — MasterBot as primary benchmark:** The target for supervised training is: the trained model, used as the evaluation function in Alpha-Beta HPS search, beats MasterBot (the Steam client's native AI). MasterBot is the strongest known Prismata AI and the most meaningful real-world reference. Access is via **SteamAI** (`js_engine/steam_ai.js`), which spawns Steam's native `PrismataAI.exe` per turn and communicates via stdin/stdout JSON. Fully integrated into the matchup runner: `node matchup_clean.js --player SteamAI --games N`. Supports parallel workers and player-switch mode.
- **V6 — Dataset composition, record scale, and supplementary data:**
 - **PvP status confirmed:** The 102,697 usable training games are all PvP — already verified. MasterBot has a rating ~ 1.0 and is excluded by the `rating >= 1500` filter, so no human-vs-bot games appear in the primary training set.
 - **Record count estimate:** At approximately 30 turns average game length, with **one record per player-turn (start-of-turn states only)**, the 102,697 games yield roughly **3.1 million training records** ($102,697 \text{ games} \times \sim 30 \text{ player-turns per game} \approx 3.1\text{M}$). Mid-turn states (after individual actions within a turn) are explicitly excluded — they introduce distribution shift, label ambiguity, and feature complexity without established benefit. **Codebase confirmation:** HPS search evaluates only end-of-turn states. `MoveIterator_PPPPortfolio.cpp` completes all 4 `PartialPlayer` phases (Defense, ActionAbility, ActionBuy, Breach) before returning child states to the search. The evaluator is never called on mid-turn positions. Start-of-turn training data matches the search's evaluation context exactly. VERIFY actual average game length from a sample of balance-validated replays; the 30-turn estimate is unconfirmed and the record count scales directly with it.
 - **"Grey data" — human vs MasterBot replays:** A potentially large pool of human-vs-MasterBot games may exist in the replay database. These are lower-confidence than PvP data: the human player is typically stronger than MasterBot (so the win-probability target is informative), but strategic intent is unknown — the human may have been experimenting, testing unusual strategies, or playing casually. This data could supplement training if the primary 103K dataset proves insufficient, but should not be mixed into the primary training set without a separate quality gate. Assess the size of this pool from the replay database and note as a contingency data source.
- **V7 — Rating distribution (measured, confirmed):** The usable training dataset (102,697 games, both players ≥ 1500 , rated) has a mean combined rating of approximately **3,909** (per-player average $\sim 1,954$). Distribution: 2000+ = 38.9%, 1800-1999 = 47.2%, 1600-1799 = 13.6%, 1500-1599 = 0.2%. Only ~ 230 games fall in the 1500-1599 bracket. This is strong news: the imitation ceiling concern is largely

addressed. The dataset is dominated by high-level play, not diluted by low-rated games. The 1500 floor adds almost no weak games. These figures are stable.

- **V8 — Unit properties available for set-based encoding:** All 105 random-pool units have structured properties in `cardLibrary.json`: buy cost (broken down by resource type), toughness, build time, default blocking, lifespan, fragile flag, and other numeric stats. These are encodeable as a fixed-length property vector per unit. Unit *abilities*, however, are defined as scripts rather than numeric values — encoding them requires either simplification (e.g. "does this unit generate attack? how much?") or omission. This is relevant to the set-based encoder approach described in Phase 1a Option 4.
- **V9 — C++ AI <-> JS engine click protocol correctness (hard gate for Phase 5 and Phase 6):** V4 verifies JS engine rule correctness; V9 verifies that the C++ AI's chosen moves translate correctly through the click protocol. Failures manifest as `skippedBuys` (clicks legal in C++ but rejected by JS) or incorrect game state after ability activation. All move types must work: BUY, USE_ABILITY (including shift-click), UNDO_USE_ABILITY, ASSIGN_BLOCKER, SNIPE/CHILL (two-step targeting), END_PHASE.

Verification protocol: Run a 512+ game LiveHardestAI vs MasterBot matchup (Phase 0b). Before Phase 5 begins, confirm:

- Zero `skippedBuys` logged across all games
- Zero click failures for all move types
- Game outcomes are plausible under human expert review

Phase 5 and Phase 6 remain hard-gated on this verification completing cleanly.

Overview

The goal is to train a neural network evaluation function for Prismata that, when used inside the existing Alpha-Beta Hierarchical Portfolio Search (HPS) engine, produces an AI stronger than MasterBot (the Steam client's native AI).

Churchill (2019) demonstrated this approach works — training on 500K AI self-play games produced an evaluator that beat both the resource heuristic and the playout evaluator. We are building on that result with three key differences:

1. **Accurate engine:** The JS engine correctly implements all game rules. Prior C++ approximations had known errors.
2. **Human replay data:** Human expert games carry stronger strategic signal than AI self-play (the AI being trained against will be stronger than prior AI self-play partners were).
3. **Architecture exploration:** Churchill's 2-layer, 512-neuron MLP was chosen without exhaustive search. We will run targeted ablations before committing to a full training run.

Self-play iteration (AlphaZero-style) is planned as a future phase. This plan covers supervised training only.

Known limitation — distribution shift: A model trained on expert human game positions will be evaluated on positions generated by AI search — which may include states humans rarely reach (deep tactical traps, unusual defensive formations, sacrifice lines). This is a fundamental limitation of supervised learning for game evaluation functions and is the primary motivation for Phase 6 (self-play iteration). Phase 5 evaluation should include qualitative review of cases where the model's evaluation appears confident but strategically wrong.

A note worth highlighting: "*The Stratego paper (Sokota et al. 2025) is probably the closest analog to what you're attempting in terms of game complexity and budget constraints — their approach of self-play RL + test-time search on an accessible budget maps almost directly to Prismata.*" We agree, and it is the primary reference for the self-play phase (Phase 6).

A Note on Code Cleanup

The `cardLibrary.json` contains 161 entries, of which 45 are deprecated, event-mode, or non-competitive units. These units never appear in balance-validated competitive replays and should not appear in any training feature vector.

Recommendation: Before finalising the feature schema, prune `cardLibrary.json` to remove the 45 non-competitive entries. This is not strictly required (the `valid_units.json` whitelist already filters them out at runtime), but it reduces confusion, eliminates a source of latent bugs, and produces cleaner code going forward. This can be done as a standalone task independently of the training pipeline.

Phase 0: Prerequisites and Baseline Measurements

Goal: Complete all setup before any training work begins. Nothing in Phase 1+ should start until all items here are done and recorded.

0a. Complete Balance Validation (DONE)

- Full re-validation of all 203,602 codes completed March 7, 2026 — single clean run with current cost+rarity validator.
- Results: 154,061 balance-passed, 48,707 failed, 834 S3 fetch errors.
- Results imported into `replays.db`. Final usable training set: **102,697 games** (both players ≥ 1500 , rated, balance-passed, all eras).
- Produce final canonical code list: `balance_validated_1500plus.json` — pending export from DB.

0b. Benchmark Tournament (TODO)

- Run 512+ game **LiveHardestAI vs MasterBot** matchup via SteamAI (`node matchup_clean.js --player-white LiveHardestAI --player-black SteamAI --games 512`). Record win rate.
- This run also serves as V9 click-protocol verification — confirm zero `skippedBuys` and zero click failures.
- Human expert visual review of a sample of games should confirm strategic coherence.
- MasterBot is the primary evaluation target for Phase 5.

0c. JS Engine Validation on Representative Sample

- Run replay validation on a fresh random sample of 1,000 balance-validated codes as a pre-extraction sanity check.
- Record pass rate.
- **Decision gate:** If pass rate is below 95%, investigate failures before proceeding. Systematic failures indicate an engine correctness issue that would corrupt training data.

- **Full validation during extraction:** During Phase 2a, every replay will be validated as it passes through the engine. Any replay that fails JS engine stepping will be logged and excluded. This makes the sample check a fast pre-flight, not the sole validation.

0d. Dataset Composition Audit

- For the final code list: measure PvP vs PvAI ratio (V6)
- Measure rating distribution (how many 1500-1799, 1800-1999, 2000+)
- Measure game length distribution (average turns per game for record count estimate)
- Measure card set distribution (base-only vs random-set games)
- **Measure first-player win rate** across the training set. If P1 or P2 win rate deviates from 50% by more than 2 percentage points, record the asymmetry for use in the Elo prior formula (Phase 1b).
- **Identify forfeit/timeout/resignation games.** Check whether the replay data distinguishes between wins by destruction (all opponent units destroyed), wins by resignation, and wins by timeout. If timeouts are common (>1% of games), exclude them. If resignations are common, note them as a potential label noise source — a player may resign a close game. **Resignation label noise mitigation:** Players resign when they *believe* they're losing, which may be 5-10 turns before actual loss. The last 3-5 positions before a resignation carry label 0 (loss) but the true win probability may be 0.15-0.30. If >5% of games end by resignation: either trim the last 3-5 pre-resignation positions from training, or flag them for a separate ablation (with/without trimming). At minimum, measure how many games end by resignation and report.
- **Measure per-unit frequency distribution** across all 102,697 games. Note any units appearing in fewer than ~500 games as "low-data units" — evaluation quality for sets containing these units will be poor. Flag these units for separate reporting in Phase 5a.
- **Turn-bucketed label entropy:** Report label entropy by turn bucket (e.g., turns 1-8, 9-18, 19+) to quantify noise level at different game phases. This informs the label strategy decision in Phase 3.
- Record all findings. They directly inform feature schema choices and training split decisions.

0e. C++ Inference Speed Benchmark (Before Architecture Commitment)

Before Phase 3 architecture decisions are finalised, benchmark how quickly the C++ `NeuralNet.cpp` inference engine evaluates positions at different model sizes. This trade-off — model quality vs. search depth — is the central tension and should be measured, not assumed.

- Benchmark: evaluate 10,000 positions with models of sizes 128h/2L, 256h/2L, 256h/3L, 512h/2L, 1024h/2L. **Also benchmark residual MLP variants:** 256h/4R, 128h/4R (4 residual blocks). Record milliseconds per evaluation.
- **Benchmark at realistic think times** (3s and 7s): measure the actual search depth achieved with each model size, not just raw evals/sec. HPS's effective branching factor determines how much depth a given eval speed buys.
- If a 1024h model takes 0.5ms but a 256h model takes 0.05ms, the smaller model gets 10x more search depth at fixed think time. At 7 seconds think time, this is likely worth more than the quality gain from a larger model.
- Record results and use them as a constraint on Phase 3 architecture selection. Any architecture too slow to evaluate at least ~500 positions/second should be deprioritised regardless of validation accuracy.

0f. Optional: cardLibrary Cleanup

- Identify and remove the 45 non-competitive unit entries from `cardLibrary.json`
- Update any code that references them (JS engine, C++ loader)
- This is independent of the training pipeline; can be parallelised with other Phase 0 work

Phase 1: Feature Schema Design

Goal: Define a canonical, versioned feature vector before any data extraction begins. This schema must be agreed upon and frozen before Phase 2 starts. Changing the schema after extraction requires full re-extraction.

1a. The Unit Representation Problem

A game of Prismata uses 11 base set units plus 8 randomly selected units from the 105-unit dominion pool — so at most 19 unit types are present in any given game. However, different games use different random units. For the feature vector to be universal (usable across all card sets), it must accommodate all 116 possible unit types, with zero-counts for types not present in the current game.

Four representation options were considered:

Option 1 — Type-Count (Churchill 2019 Baseline)

For each of the 116 unit types, record how many units of that type each player owns. Add resource counts and the current player indicator.

- **Feature vector size:** 116×2 (count per player) + 12 (resources: 6 types $\times$ 2 players) + 1 (player to move) = **~245 features**
- **Advantages:** Simple, fast inference, established precedent. Churchill achieved competitive results with this approach.
- **Disadvantages:** Loses all instance-level information: which units are under construction, which have been used this turn, which are currently blocking, which have taken partial damage.
- **Churchill's note:** "This encoding discards information such as which units may be activated, individual unit instance properties... since the states are all recorded at the beginning of each turn when units are not yet activated, much of the effect of this information loss is alleviated."

Option 2 — Type-Count with Instance Flags, Supply, and Card-Set Indicator (Selected)

Extend Option 1 with per-type aggregate flags, supply tracking, and a card-set presence indicator. This matches the feature layout already implemented in `NeuralNet.cpp`.

For each of the 116 unit types, encode **11 values**:

- P0 ready count, P0 constructing count, P0 exhausted/ability-used count, P0 blocking count
- P1 ready count, P1 constructing count, P1 exhausted/ability-used count, P1 blocking count
- P0 supply remaining, P1 supply remaining
- In card set (1.0 if this unit type is available for purchase in this game, 0.0 otherwise)

Plus **14 global features**:

- P0 resources (gold, blue, red, green, energy, attack) — 6 values
- P1 resources (gold, blue, red, green, energy, attack) — 6 values
- Turn number (normalized)
- Active player indicator (0 or 1)
- **Feature vector size:** $116 \times 11 + 14 = 1,290$ features
- **Advantages:** Captures tactically relevant state including construction status, ability usage, blocking assignments, supply exhaustion, and card-set composition. The `in_card_set` flag lets the model distinguish "not in this game" from "in this game but currently unbought." Supply remaining is strategically critical — knowing the opponent has bought 3 of 4 available Tarsiers is qualitatively different from knowing they've bought 3 of 20.
- **Disadvantages:** ~5x larger than Option 1; more expensive to extract and slower to infer (though still well within C++ inference speed requirements at this scale).
- **Why this is selected:** The JS engine reliably exposes all these per-instance fields. Using them is the primary advantage of the accurate JS engine over the prior C++ approximation. **The C++ inference engine already implements this exact layout** (`NeuralNet.cpp:266-388`) — no C++ changes needed. Run Option 1 as a Phase 3a baseline for comparison.

Option 3 — Per-Instance Encoding — Maximum expressiveness; variable-length input. Future experiment if Options 1/2 plateau.

Option 4 — Set-Based / Property Encoding — Encode unit *properties* rather than identity. Generalises across card sets. Promising medium-term direction; requires ability schema design. See V8.

1b. Label Design

- **Training target:** probability that the active player wins the game from this state
- **Raw label:** 1 if active player won, 0 if lost (draw = 0.5)
- **Output activation:** sigmoid (range 0-1) or tanh (range -1 to 1) — both work; Churchill used tanh.
Note: The existing C++ inference engine (`NeuralNet.cpp:531-535`) uses `tanhf()` with output range $[-1, 1]$, scaled by $\times 100$ in `Eval.cpp` for alpha-beta compatibility. Training must match: either use tanh output (matching existing C++) or update `Eval.cpp` if switching to sigmoid. Standardise and document before training.
- **Loss function:** Binary cross-entropy (BCE) on value prediction. BCE is the theoretically correct proper scoring rule for probability estimation and handles soft labels in $[0.3, 0.7]$ more naturally than MSE. Churchill found MSE and BCE produce similar results with hard labels, but with soft labels BCE is the principled default. **Run MSE and Huber loss as Phase 3 ablations** to verify. Huber caps gradient for outlier labels (e.g., player resigns a winning position — label 0 for an objectively strong state), reducing sensitivity to mislabeled games.
- **Implementation:** Use `torch.nn.BCEWithLogitsLoss` (not `BCELoss` after manual sigmoid). The `WithLogits` variant is numerically stable for soft targets and avoids $\log(0)$ when predictions are near 0 or 1.

- **Primary offline metric:** BCE loss on the temporal validation set is the primary metric for comparing ablation runs. Report Brier score as a secondary diagnostic.

The Noisy Label Problem (Critical)

Binary game outcomes are a very noisy signal for early-turn positions. A turn-1 position from a game Player 1 eventually won gets label 1.0, but the true win probability at turn 1 is close to 0.5 for both players. The model must average out this noise, and it is working from a very poor signal-to-noise ratio in the opening and midgame — precisely where evaluation quality matters most for search.

This problem is more acute for high-rated human games than for Churchill's AI self-play. AI self-play games are shorter and more decisive (errors compound faster at lower skill). Human 2000-rated games are longer and closer, meaning the true win probability drifts slowly from 0.5, making the binary label noisier for more turns.

Label strategy: 3-way ablation (Phase 3)

The following three label strategies will be compared in Phase 3 ablations. Do not commit to one before empirical results:

Strategy A — Hard binary labels (baseline) Use raw 0/1 labels. Simplest, no assumptions. The model must average out all noise itself.

Strategy B — Hard labels with temporal sample weighting Keep 0/1 labels but weight each sample's loss contribution by game progress:

```
REFERENCE_LENGTH = 40 # approximate 75th percentile game length
t = min(1.0, turn_number / REFERENCE_LENGTH)
sample_weight = 0.3 + 0.7 * t # early positions contribute less to loss
```

This downweights noisy early positions without modifying the labels themselves. Lower ceiling than Strategy C but requires zero label schema changes.

Strategy C — Elo-interpolated temporal labels Blend the game outcome with an Elo-based prior that decays over the course of the game:

```
elo_prior = 1 / (1 + 10 ** ((opponent_rating - player_rating) / 400))
REFERENCE_LENGTH = 40 # fixed constant, NOT total_turns
t = min(1.0, turn_number / REFERENCE_LENGTH)
label = (1 - t) * elo_prior + t * actual_outcome
```

The interpolation parameter `t` uses a **fixed reference length** (approximately the 75th percentile game length) rather than `total_turns`. This eliminates outcome-correlated information leakage — using `total_turns` would encode game length, which is correlated with how decisive the game was (short games = stomps, long games = close).

Implementation note: If Phase 0d reveals a first-player advantage >2pp, adjust the Elo prior for equal-rated games accordingly (e.g., if P2 wins 53%, use `elo_prior = 0.53` for P2 at equal ratings instead of 0.50).

Non-linear ramp variant (sub-ablation within Strategy C): The first 10 turns of a Prismata game reveal less about the winner than the last 10. A non-linear ramp concentrates label authority in the late game:

```
# Quadratic ramp — early labels stay close to Elo prior longer
t_raw = min(1.0, turn_number / REFERENCE_LENGTH)
t = t_raw ** 2 # quadratic: slow start, fast finish
label = (1 - t) * elo_prior + t * actual_outcome
```

Test quadratic (`t**2`) alongside linear in the Phase 3 label ablation. If the difference is negligible, use linear (simpler).

- *TD(λ) bootstrapping* (higher ceiling, more complex): use an initial model's own predictions to generate soft labels for a second pass. Standard in game AI but requires an iterative training loop. Consider for self-play iteration 2+ (Phase 6).

1c. Schema Versioning

- Schema stored as a single versioned JSON file: `training/schema_v1.json`
- Must include:
 - Unit list (ordered)
 - Feature names and indices
 - Value ranges
 - **Normalization method and constants** — the C++ inference engine uses `clamp_divide` normalization with specific per-feature caps:
 - Gold: /20, Blue: /5, Red: /5, Green: /15, Energy: /10, Attack: /25, Turn: /30
 - Unit counts: raw (no normalization — typically 0-20 range)
 - Supply remaining: raw
 - Binary flags (`in_card_set`): 0 or 1
 - The JS extraction pipeline must apply **identical normalization constants**. Mismatches produce silently wrong evaluations in C++.
 - Label encoding (which strategy from Phase 1b was selected)
- Any change to the feature vector = new version. Old data cannot be mixed with new schema.
- The canonical unit ordering comes from `valid_units.json` (base set first, then random set alphabetically, or by some consistent ordering). This ordering must be fixed and documented.
- **Include a schema hash/checksum in output filenames and binary shard headers** to prevent accidental mixing of schema versions. Filenames can be renamed; headers can't.

Phase 2: Data Extraction Pipeline

Goal: Extract all usable human replay data through the accurate JS engine into a single canonical dataset. All previous JSONL files are considered legacy and are not mixed into this output.

2a. Extraction

- **Input:** `balance_validated_1500plus.json` from Phase 0a
- **Process:** For each replay code, fetch from S3, step through game using JS engine, record **one training record per player-turn at start-of-turn states only**. Mid-turn states (after individual actions within a turn) are excluded — they introduce policy-dependent distribution shift, label ambiguity, and feature complexity.
- **Output:** Binary shard format (matching the existing selfplay pipeline structure for loader compatibility), or HDF5 with chunked datasets. JSONL at 3.1M records (~15-29 GB) is impractically slow to load.
- **Record metadata:** Each record must include: `replay_code`, `turn_number`, `rating_p0`, `rating_p1`, `game_date`, `card_set` (the 8 random unit IDs present in this game). These enable temporal splitting, stratified analysis, and post-hoc filtering without re-parsing replays.
- **Include `total_turns`** in each record: required to compute Elo-interpolated labels (Phase 1b Strategy C) at training time without re-parsing replays. Note: `total_turns` is used only for label computation and is **never** an input feature to the model. **Hard requirement:** Store metadata (`total_turns`, `rating_p0`, `rating_p1`, `game_date`, `replay_code`) in a separate metadata block from the feature vector. Add an extraction test that verifies no metadata field appears in the feature vector indices.
- **Note:** Store raw state fields alongside the feature vector in each record if feasible. This allows re-vectorisation (re-running the featurisation step without re-fetching replays) if the schema changes during Phase 3 ablations.
- **Engine validation during extraction:** Validate every replay as it passes through the JS engine. Log any replays that fail engine stepping (invalid moves, state inconsistencies). If the failure rate exceeds 1%, halt and investigate before proceeding. This makes V4 verification comprehensive rather than sample-based.
- **Parallelization:** Each replay is independent. Use 4-8 worker processes to reduce extraction time from ~14 hours to ~2-4 hours.

Symmetry augmentation (free 2x data):

Prismata is symmetric between players. For every training record from Player 0's perspective, generate a mirror record from Player 1's perspective: swap all P0/P1 features and invert the label ($1 - \text{label}$). This doubles the effective dataset at zero additional extraction cost.

For this to work, the feature vector must be structured so the swap is trivial. **Requirement:** design the Phase 1 schema with P0 features in a contiguous block followed by P1 features in the same layout. The mirror record is then a single slice-and-swap operation. Document this requirement explicitly in `training/schema_v1.json`.

Note on symmetry augmentation scope: With the canonical P0/P1 encoding (confirmed in `NeuralNet.cpp`), symmetry augmentation generates genuinely new training examples — the model sees the same position from both players' perspectives. This is a true 2x data increase, not redundant with active-player-relative encoding.

Base-only game exclusion:

Base-only games (no random units — both players use only the 11 base-set cards) are strategically very different from the random-set format that constitutes all competitive play. Including them introduces a distribution the model must learn that has near-zero relevance to the target deployment context. Exclude base-only games from the primary training set. Measure their count during Phase 0d so the exclusion can be quantified.

2b. Train / Validation / Test Split

- **Primary split: temporal.** Sort all replays by game date. The most recent 10% of games = test set, the next most recent 10% = validation set, the remaining 80% = training set. This better reflects deployment conditions: the model will face positions generated by current-era play, not a random sample of historical play. Prismata's strategic meta evolved over time — opening strategies, unit valuations, and player styles have shifted.
- Within each temporal partition, no further splitting is needed — replay-code-level integrity is maintained by the temporal ordering.
- **Secondary diagnostic: random split.** Also compute validation metrics on a random 10% holdout to compare against the temporal split. If temporal validation loss is much worse than random, the model is memorizing historical patterns rather than learning general strategy.
- Test set held out entirely — not used during Phase 3 architecture search.
- **Symmetry augmentation is applied AFTER the split.** A mirror record from a test game must never appear in training. Assign each replay code to a split first, then generate mirror records within that split.

2c. Deduplication

- Some codes may appear across multiple source lists. Deduplicate by code before extraction begins.
- After extraction, verify no duplicate records (same replay_code + turn combination)

2d. Post-Extraction Validation

- Spot-check 100 random records against expected game outcomes
- Verify label distribution (~50/50 expected given symmetrically-rated games)
- Verify feature value ranges (no NaN, no Inf, no obviously wrong counts)
- Verify total record count is consistent with game length estimate
- **Mirror augmentation correctness test:** For a sample of 1,000 records, verify:
 - `mirror(mirror(record)) == record` (double-mirroring recovers the original feature vector exactly)
 - `label_original + label_mirror == 1.0 exactly` (labels invert correctly) — this must hold for all three label strategies including Strategy C (Elo-interpolated). For Strategy C, verify algebraically: if `label = (1-t)*elo_prior + t*outcome`, then `mirror_label = (1-t)*(1-elo_prior) + t*(1-outcome) = 1 - label`.
 - All player-specific features (unit counts, resources, supply) swap correctly between P0 and P1 blocks
 - The active player indicator flips
 - This catches subtle asymmetry bugs in the feature extraction that would silently poison the dataset.

Phase 3: Architecture Search and Ablation

Goal: Determine a good architecture before a full training run. Use a representative subset of training data. Do not proceed to Phase 4 without reviewing results here.

This phase is the primary place where the plan should be adjusted based on empirical findings. **Track ablation runs with W&B or MLflow** — with 15+ configurations across architecture, features, labels, and regularization, spreadsheet tracking becomes error-prone.

3a. Churchill Baseline (Lower Bound)

Replicate Churchill (2019) as closely as possible on our data:

- Feature set: Option 1 (type-count, ~245 features)
- Architecture: 2 hidden layers, 512 neurons, tanh activation
- Optimiser: Adam, LR = 1e-5
- Loss: MSE, value prediction only

This is the lower bound. If a more complex model cannot beat it, something is wrong with the model, the data, or the problem formulation.

3b. Feature Richness Ablation

Train same Churchill architecture on Option 2 features (~1,290 features) and compare validation loss.

- If Option 2 adds less than ~1 percentage point: determine whether the complexity is justified
- If Option 2 adds ≥ 2 percentage points: Option 2 is likely the better starting point

3c. Capacity Search and Architecture Variants

Vary hidden dimension: 128, 256, 512, 1024. Vary depth: 2, 3, 4 layers. Use the winning feature set from 3b.

Key question: is 512 neurons (Churchill's choice) actually optimal, or is it over/under-powered for a ~245 or ~1,290-feature input? A smaller model infers faster in C++ (directly increases search depth at fixed think time). A 256-neuron model may be preferable to a 512-neuron one if accuracy is similar. Cross-reference against the Phase 0e inference speed benchmark.

Residual MLP blocks: Include at least one run with skip connections (residual blocks) alongside the plain MLP:

```
class ResBlock(nn.Module):
    def __init__(self, dim):
        super().__init__()
        self.fc1 = nn.Linear(dim, dim)
        self.fc2 = nn.Linear(dim, dim)
        self.norm = nn.LayerNorm(dim)
    def forward(self, x):
        return x + self.norm(F.relu(self.fc2(F.relu(self.fc1(x)))))
```

Skip connections allow 4-6 block depth without degradation and are trivially portable to C++. **The C++ inference engine already supports residual blocks with LayerNorm** ([NeuralNet.cpp:146-154, 442-459](#)). They do not add significant inference overhead at these model sizes. Compare a 4-block residual MLP against the plain MLP winner from the depth/width search.

Ablation efficiency: Do not train each ablation to convergence. Use a **fixed wall-clock time budget** (e.g., 2 hours per ablation) rather than a fixed epoch count. This is fairer — 15 epochs of a 1024-neuron model costs ~8x more than 15 epochs of a 128-neuron model, conflating model speed with model quality. Wall-clock budgets measure what matters: relative architecture quality at the same compute cost. Compare validation loss curves at the budget endpoint.

3c-2. Label Strategy Ablation

Compare the three label strategies from Phase 1b on the best architecture from 3c:

1. **Strategy A:** Hard binary labels (0/1)
2. **Strategy B:** Hard labels + temporal sample weighting
3. **Strategy C:** Elo-interpolated labels with fixed reference length

Use the same training budget and hyperparameters for all three. Compare validation loss (and calibration if feasible). The winner becomes the label strategy for Phase 4.

3d. Policy Head Experiment (Recommended Ablation)

Add a policy head predicting which move the human took. Even weak policy accuracy (20-25%) can significantly improve Alpha-Beta search quality via PUCT-style move ordering — this is especially valuable in games with Prismata's extreme branching factor. The C++ PUCT infrastructure already exists ([UCTSearch.cpp](#)).

- If move-prediction accuracy reaches ~20%+, include the policy head in the Phase 4 full training run.
- If accuracy is below ~15%, the signal is too weak to help.

3e. Architecture Decision Gate

Before Phase 4, review:

- Which feature set won (Option 1 vs 2)?
- What is the optimal depth and width?
- Does validation accuracy plateau before the full dataset? (If yes: data-limited, not capacity-limited — consider self-play data generation sooner)
- Is a policy head worth including?
- Which label strategy won (A, B, or C)?

Decision rule for ambiguous results: If the feature-richer model loses more than 0.1ms/eval vs the simpler model (from Phase 0e) but improves validation loss by less than 0.005, prefer the simpler model — inference speed dominates at fixed think time. If results are genuinely tied, prefer the simpler architecture.

Phase 4: Full Training Run

Goal: Train the chosen architecture from Phase 3 on the full training set.

4a. Training

- Full training set (80% of extracted records, per temporal split)
- Validate every epoch on validation set
- Save best checkpoint by validation loss
- **Gradient clipping:** Apply gradient clipping (max norm 1.0) as a default for all training runs. This prevents occasional exploding-gradient training collapses when the model is confident but wrong (predicted 0.95, label is 0.0), which is common with BCE loss and noisy binary labels. Never harmful, cheap insurance.

- **Stochastic Weight Averaging (SWA):** Maintain an SWA model alongside the standard checkpoint — average weights from the last ~20% of epochs. SWA often produces smoother, more robust models with noisy labels. PyTorch provides `torch.optim.swa_utils`. Start SWA collection relatively late — starting too early while loss is still rapidly descending washes out the weights. Compare SWA model against best single checkpoint in Phase 5.
- Early stopping with patience ~10 epochs

Rating-based sample weighting: Rather than a hard rating floor, weight each sample by the combined rating of both players. This lets the 2200-rated games dominate gradient updates while still retaining all 102,697 games. Suggested formula:

```
weight = ((p0_rating + p1_rating) / 4000) ** 2
# 2000+2000 -> weight 1.00; 1800+1800 -> weight 0.81; 1500+1500 -> weight 0.56
```

This is strictly better than a hard cutoff — keeps all data but lets high-rated games dominate. Pass as `sample_weight` to the loss function. Compare vs. unweighted baseline in Phase 3. **Note:** With 98.8% of games having combined rating ≥ 3200 (weight ≥ 0.64), the effect is mild — the natural distribution already heavily favors high-rated games. The ablation may show negligible difference, in which case drop the weighting for simplicity.

4b. Regularisation

With ~3.1M records (~6.2M after symmetry augmentation) vs Churchill's 15M, overfitting risk is higher. Apply:

- Dropout: 0.1-0.3 on hidden layers
- Weight decay: L2 regularisation (1e-4 to 1e-5)
- **Label smoothing:** Apply only if Phase 3 ablation selects Strategy A (hard binary labels). If Strategy B or C is selected, label smoothing is redundant — both already soften targets. Applying both risks over-smoothing, especially for late-game positions where hard outcomes are the correct signal.
- **Mixup augmentation:** Interpolate pairs of training records and their labels to create synthetic training examples:

```
lam = np.random.beta(0.2, 0.2)
x_mix = lam * x1 + (1 - lam) * x2
y_mix = lam * y1 + (1 - lam) * y2
```

This is a cheap, well-supported regularization technique for tabular data. Apply on-the-fly in the data loader (zero storage cost). Use a low alpha (0.1-0.2) to keep interpolated states plausible. Compare with and without in Phase 3.

Specific values should be set based on Phase 3 observations.

4c. Learning Rate Schedule

Churchill used a fixed Adam LR of 1e-5. Cosine decay or step decay may improve convergence — worth a quick comparison during Phase 3 before locking in.

Learning rate warmup: Use a short linear warmup (500-2000 steps) before reaching peak LR. This prevents early training instability with Adam + BCE on noisy labels, especially if gradient clipping is relaxed. Standard practice, cheap insurance.

4d. Infrastructure

- Local: Intel Arc B580 (12GB VRAM, XPU backend)
- Cloud: AWS g4dn.xlarge or GCP g2-standard-4 if local is insufficient
- Monitor training vs validation curves throughout; any run where training accuracy significantly exceeds validation accuracy early should be investigated before completing

4e. Export and C++ Integration

- Export weights to binary format compatible with `NeuralNet.cpp`
- Verify all expected tensors are present (26 tensors required by C++ loader: input_proj + N x (linear1, norm1, linear2, norm2) + policy(2) + value(2))
- Smoke test: load weights in C++, evaluate a known state, confirm output is in expected range
- **Cross-language verification:** Evaluate the same canonical game state in Python and in C++. Confirm outputs match to 4+ decimal places. **Cast PyTorch model and dummy inputs to `torch.float32` before export** — C++ uses 32-bit `float` throughout (`NeuralNet.cpp`), while Python/PyTorch defaults to 64-bit. Precision drift from float64→float32 truncation will cause spurious mismatches in the 4-decimal verification.
- **Export normalization constants** alongside weights in `schema_v1.json`. The C++ inference engine hardcodes normalization caps — verify they match the extraction pipeline's values.
- **Eval score calibration for alpha-beta:** The C++ search framework uses different score scales for different evaluation methods. Pure NeuralNet evaluation (`Eval.cpp:77-78`) scales by $\times 100$ to $[-100, +100]$. Playout evaluation returns $[-10000, +10000]$. The blend mode (`NeuralNetPlusPlayout, Eval.cpp:226-239`) correctly scales NN output by $\times 10000$ to match Playout range. **Action required:** When deploying the trained model, test both pure NeuralNet mode ($\times 100$ scale) and NeuralNetPlusPlayout blend mode. The compressed $\times 100$ range may produce different search dynamics than the $\times 10000$ range. If pure NeuralNet is preferred, consider updating the $\times 100$ factor in `Eval.cpp` to $\times 10000$ for better alpha-beta compatibility. Document the chosen mapping.

Phase 5: Evaluation

Goal: Measure trained model strength against MasterBot and assess whether success criteria are met.

5a. Tournament Setup

- Primary opponent: **MasterBot** (Steam) via SteamAI.
- Games: **minimum 1,024** for the primary evaluation. At 1,024 games, a 3 percentage point win rate difference is detectable at 95% confidence. If the result is borderline (e.g., 52% win rate, CI barely excludes 50%), run an additional 512-1,024 games to confirm.
- Card sets: random card sets matching normal play conditions. **Use paired matches:** for each card set, play two games with swapped starting players. This halves variance from card-set and first-player effects, making win-rate differences more detectable at the same game count.

- Think time: **3 seconds per turn** (consistent with Phase 0b benchmark). **Also report results at 1s and 7s think times** — if the neural player's advantage increases with think time, the evaluator is good but needs search depth to express it. If it decreases, the evaluator may be brittle under deeper search.
- Report Wilson confidence interval, not just point estimate. **Use paired statistical tests** (McNemar's test or sign test on pair outcomes) rather than simple binomial, since matches are paired by card set.
- **Turn-bucketed evaluation:** Report validation loss and prediction accuracy by turn quartile (opening: turns 1-8, mid-game: turns 9-18, late-game: turns 19+). A model that looks fine overall may fail badly in early or late game.
- **Card-set stratified evaluation:** Report tournament win rate separately for: (a) card sets that appear in training data vs. not seen, (b) card sets containing low-frequency units (flagged in Phase 0d), (c) exact 8-unit sets seen in training vs. unseen sets but seen units vs. rare-unit-heavy sets.
- **Every primary tournament result must note whether the opening book was active.** Run at least one evaluation with and one without.

5b. Success Criteria

- **Pass:** Win rate vs MasterBot above 50% with 95% confidence (lower bound of Wilson CI > 0.50)
- **Partial pass:** Above 50% but not yet statistically significant — run more games
- **Fail:** Below 50% — investigate data pipeline and Phase 3 findings before concluding model is fundamentally too weak
- **Note:** If the model loses to MasterBot but the margin is small (e.g., 45-48% win rate), that's a partial success indicating the approach works but needs more data or self-play iteration.

5b-2. Offline Search-Position Sanity Check (Before Tournaments)

Before running expensive tournament games, verify the model behaves sanely on positions generated by AI search (not just human replay positions):

- Run HPS + new eval on a batch of ~1,000 self-generated positions from search
- Check value consistency: positions that are objectively similar should have similar values
- Check mirror symmetry: `value(state) + value(mirror(state)) ~= 1.0`
- Check tactical sanity: does the model correctly prefer obviously winning positions?
- **Value monotonicity probes:** Verify that adding significant free resources to the active player doesn't dramatically reduce the predicted value (a small drop is fine due to edge cases, but a large swing like 0.7→0.2 indicates a feature bug). Similarly, removing opponent blockers shouldn't dramatically reduce value. These are soft sanity checks, not strict invariants.
- If the model produces erratic values on search-generated positions while performing well on validation, this signals distribution shift — the primary supervised learning limitation

This is a fast diagnostic (~10 minutes) that can save hours of inconclusive tournament games.

5b-3. Search Integration Ablation

The C++ engine supports multiple evaluation modes. Compare tournament results across:

1. **Pure NeuralNet** — neural evaluation only (`EvaluationMethods::NeuralNet`)
2. **NeuralNetPlusPayout** — weighted blend of neural + payout evaluation
3. **Pure Payout** — baseline (existing `OriginalHardestAI` behavior)

The blend mode may outperform pure neural evaluation, especially if the neural model has weak spots on certain position types. The blend weight is configurable in the AI parameters JSON. Test at least 3 blend weights (e.g., 0.3, 0.5, 0.7 neural weight).

5c. Secondary Evaluations

- Run model vs itself (expected ~50%, confirms basic sanity)
- Run vs LiveHardestAI as a stable C++ reference for future comparability
- If Prismata players are available, qualitative play sessions against the model are valuable for assessing strategic coherence that statistics may miss
- **Value prediction accuracy on held-out test set:** For test set positions, how well does the model's predicted win probability match the actual game outcome? Report calibration plots (predicted probability vs actual win rate, binned by decile).
- **Move agreement rate (if policy head is included):** How often does the neural-guided AI choose the same move as expert human players in the same position? This measures policy alignment independent of search.
- **Value symmetry sanity check:** For a sample of test positions, verify `value(state) + value(mirror(state)) ≈ 1.0`. If this fails, there is a bug in features or mirroring.

5d. Decision Gate: Proceed or Iterate?

- **Success:** Proceed to Phase 6 (self-play iteration)
- **Failure:** Diagnose root cause. Return to Phase 3 (architecture) if model architecture is suspected; Phase 1 (feature schema) if features seem inadequate; Phase 2 (data quality) if the pipeline seems suspect.
- **Partial success:** Begin self-play data generation with current model and assess whether one iteration of self-play improves it before fully rearchitecting.

Opening Book

Current State

The HPS search engine uses an opening book to bypass search for the first few turns, playing pre-computed strong sequences instead. The current book has two tiers, both extracted from the live game's AI parameters (`Prismata.swf`):

- **LiveOpeningBook** — 4 general entries: default sequences for when no specific unit triggers apply (e.g., "if Vivid Drone is in the set, open with X")
- **LiveOpeningBook2** — 50 state-specific entries covering 9 random-pool units deemed to have a high influence on opening turns (Wild Drone, Doomed Wall, Energy Matrix, Vivid Drone, Galvani Drone, Centurion, Doomed Drone, Xeno Guardian, Defense Grid). Multiple entries per unit handle different board states (P1 vs P2 starting positions, combinatorial pairings).

These were hand-coded by Lunarch Studios and reflect the meta-knowledge of professional game designers. Configuration: `bin/asset/config/config.txt` under `LiveOpeningBook` and `LiveOpeningBook2`.

Opportunity: Data-Driven Opening Book from Expert Replays

The 102,697 expert replays provide a much larger and more empirically grounded source for opening book extraction. For each card set, we can observe what 2000+ rated players actually do on turns 1-3. The existing `training/opening_book.py` was written for a previous attempt at this and may be adaptable.

Approach:

- Group replays by which random units are present in the card set
- For each unit (or combination of units), extract the turn-1 and turn-2 buy decisions from games in the top rating bracket (2000+)
- Find the most common buy sequence(s) for each triggering unit
- Generate opening book entries in the format already used by `config.txt`

Why this is valuable:

- Covers all 105 random-pool units, not just the 50 in the current hand-coded book
- Reflects actual high-level play rather than designer intuition
- Can be extracted independently of the neural training pipeline — a pure combinatorial analysis of opening data
- Strong openings reduce the importance of the early-game evaluation function, where the noisy label problem (Phase 1b) is worst

Risks and considerations:

- Opening book data should be extracted from the **training set only** — not from validation or test games. The validation/test split must be applied before extraction begins.
- An opening book biases the AI toward specific positions early in the game. If those positions are exactly the ones represented in training data, this could cause overfitting to the opening without improving evaluation quality more broadly. Monitor whether tournament performance depends on opening-book-covered card sets.
- Some card sets may have no clear consensus opening (multiple good lines exist). In those cases, it may be better to omit a book entry and let search find the move rather than forcing a suboptimal commitment.

Relationship to neural training: The opening book and the neural evaluation function are independent. Either can be improved without changing the other. However, a strong opening book effectively reduces the turns that the neural evaluator must handle well, which may make early-game label noise less consequential in practice.

Recommended action: Extract a data-driven opening book as a standalone Phase 0 task (before Phase 5), independent of the neural training schedule. **Every primary tournament result in Phase 5 should report whether the opening book was active.** Run at least one evaluation with and one without the expanded book. If the win rate difference is large (>5pp), the model may be partially dependent on the book to funnel play into familiar positions rather than evaluating well generally.

Phase 6: Self-Play Iteration (Future Phase — Outline Only)

Documented here so the supervised training plan can be evaluated for compatibility with the self-play phase that follows.

6a. Self-Play Data Generation

- Use trained model as evaluation function in the JS engine matchup runner
- Generate a target of at least 100,000 self-play games (roughly matching human dataset size) before retraining
- Store in same schema format as human replay training data for compatibility

6b. Iterative Training

- Retrain on a mix of human replay data + self-play data
- Maintain human replay data in the mix throughout (never below 20% human data, following AlphaZero practice)
- Evaluate each iteration against its predecessor and against MasterBot

6c. Gumbel AlphaZero Consideration

The Gumbel AlphaZero paper (Danihelka et al., ICLR 2022) proposes improved exploration during self-play by sampling actions without replacement, providing policy improvement guarantees even with few simulations. This is directly applicable to the HPS search framework and should be evaluated for self-play iteration 2+.

Known Failure Modes

Document known failure modes and their detection/mitigation strategies. If any of these occur during training, consult this section before debugging blindly.

Failure Mode	Detection	Mitigation
Mode collapse — model outputs constant ~0.5 for all positions	Validation loss plateaus immediately; prediction histogram shows narrow spike	Check label distribution, verify gradient flow, check for normalization bug
Overfitting to early game — model performs well on turn 1-10 but poorly on turn 20+	Turn-bucketed validation loss (Phase 5a) shows diverging performance by game phase	Re-weight training samples toward late game, or use temporal sample weighting (Strategy B)
Card set memorization — model learns unit-specific patterns but doesn't generalize to unseen card combinations	Card-set stratified evaluation (Phase 5a) shows large gap between seen and unseen sets	Consider property-based encoding (Option 4) in future iteration; add more diverse self-play data
Calibration collapse — predictions are accurate on average but badly calibrated (always ~0.52 or always >0.8)	Calibration plots show poor alignment; Brier score diverges from log-loss	Add calibration term to loss; use temperature scaling post-hoc; verify BCE loss is active
Distribution shift failure — model evaluates human-like positions well but produces unreliable values for AI search positions	Tournament win rate is low despite good validation metrics	This is the fundamental supervised learning limitation. Proceed to Phase 6 (self-play).

Resolved Design Questions

1. **Feature richness:** Option 2 (type-count + instance flags + supply + card-set indicator) is the starting point. Option 1 (Churchill-exact) runs as a Phase 3a baseline.
2. **Rating floor:** Keep the 1500 floor with rating-based sample weighting (Phase 4a). V7 data shows only 0.2% of games (~230) fall in the 1500-1599 bracket — raising the floor gains almost nothing and loses real data.
3. **Card set filtering:** Exclude base-only games. Base-only Prismata is a strategically distinct subgame with no relevance to competitive play.
4. **Value target:** Binary win outcome with soft-label modification (Phase 1b). Win prediction is correct — soft-labelling fixes the noise issue.
5. **Architecture prior:** Residual MLP blocks allow going deeper without degradation and are trivially portable to C++. Plain MLP is fine at 2-3 layers; residual preferred at 4+.
6. **Benchmark calibration:** MasterBot (Steam) is the primary benchmark, accessed via SteamAI. Beating a fixed AI is necessary but not sufficient — supplement with held-out prediction accuracy and move agreement metrics (Phase 5c).

What This Plan Does Not Cover

- Specific hyperparameter grid search methodology (to be detailed after architecture is selected in Phase 3)
- Cloud compute provisioning specifics (to be planned once data scale is confirmed in Phase 0)
- C++ NeuralNet integration changes (existing infrastructure reused; only weight file format must remain compatible)
- Multi-GPU training (not needed at this data scale)
- Policy learning beyond move ordering in search

Deferred Enhancements

The following items were suggested during meta-review and have merit but are deferred to iteration 2:

- **Pairwise/ranking auxiliary loss** — within same game, winning-player states should rank above losing-player states. Improves search ordering. Medium effort; consider after initial tournament results.

References

- Churchill, D. & Buro, M. (2015). *Hierarchical Portfolio Search: Prismata's Robust AI Architecture for Games with Large Search Spaces*. AIIDE 2015.
- Churchill, D. (2019). *Machine Learning State Evaluation in Prismata*. AIIDE Workshop 2019.
- Danihelka, I. et al. (2022). *Policy Improvement by Planning with Gumbel*. ICLR 2022.
- Hubert, T. et al. (2021). *Learning and Planning in Complex Action Spaces (Sampled MuZero)*. ICML 2021.

- Landers, M. et al. (2025). *SAINT: Attention-Based Policies for Discrete Combinatorial Action Spaces*.
- Sokota, S. et al. (2025). *Superhuman AI for Stratego Using Self-Play Reinforcement Learning and Test-Time Search*.